

Capstone Projects

KICKOFF: FRI 5 JUN 2026 · FINAL SUBMISSION: SUN 28 JUN 2026

JUN 2026

HOW THIS WORKS

- **Pick one project** from the five below and build it end-to-end. **Solo build** — no teams.
- Every project requires real backend code, a real database, and a working frontend. Anything beyond what was taught in class has a *linked reference doc* inside the project.
- **Stack freedom** — see the toolbox panel below. Mix and match within what you've been taught. Don't reach for a tool you've never touched unless its reference doc is linked from your chosen project.
- **Free / very-low-cost tools only.** Use OpenAI / Groq / Hugging Face free tiers, Ollama locally, or Gemini Flash. Render / Railway free tier for deployment.
- You're free to **extend or deviate** from the suggestions — keep the core problem intact, then go further. The "open-ended stretch" section in each project is where to start experimenting.

KEY DATES

- **Fri 5 Jun 2026** — Kickoff: pick your project and lock your choice via the project-selection Google Form.
- **Every week** — Doubt-resolution session with your TA. Bring blockers, code, screenshots.
- **Sun 28 Jun 2026, EOD** — Final submission deadline.

WHAT TO SUBMIT

- **Project video** — a screen recording walking through your working app, **under 5 minutes** (Loom, OBS, or any screen recorder).
- **Presentation** — slides explaining the project (problem, who it's for, architecture diagram, what works, what's next, what you'd build if you had two more weeks). Gamma, Canva, or Google Slides.
- **Frontend** — submit *either*:
 - **Public URL** of your deployed app (Streamlit Community Cloud / Render / Vercel), *or*
 - **GitHub repo link** with a clear README explaining how to run it locally.
- **Backend** — submit *any one* of:
 - **Public URL** of your deployed FastAPI app + `/docs` Swagger link working
 - **GitHub repo** with backend code, `requirements.txt`, `.env.example`, and an Alembic migrations folder if you used a relational DB
- **Database** — a brief schema diagram (DBdiagram / dbml / hand-drawn) or the SQLAlchemy `models.py` file linked from your README.
- **Frontend mockups** (*optional*) — Figma / Stitch / sketches you used while building.

WHERE TO SUBMIT

- A dedicated **LMS assignment** will be released 1 week prior to the deadline of the capstone project. All deliverables above (video, presentation, frontend, backend, DB schema, optional mockups) get uploaded there.
- Submission window closes **Sun 28 Jun 2026, EOD**.

YOUR TOOLBOX (EVERYTHING YOU'VE USED IN CLASS)

Compose your stack from this list. Each layer below shows the most-used option first, with alternatives. If you mix tools, make sure the pieces talk to each other (FastAPI to Streamlit needs HTTP requests, FastAPI to vanilla JS needs CORS — see [ref-fastapi-cors-fetch.pdf](#)).

FRONTEND

Streamlit — fastest path for chat / dashboard / file-upload UIs. You built a Groq chatbot in M4.

Vanilla HTML + CSS + JS — you built the "Task Flow" Kanban with Flexbox + localStorage in M2. Use `fetch()` to call FastAPI (ref doc inside).

BACKEND

FastAPI — Pydantic v2 models, Depends , OAuth2PasswordBearer , HTTPException . You built a full CRUD + JWT API in M3. Use **Uvicorn** with `--reload` in dev.

DATABASE (RELATIONAL)

SQLite for solo dev — zero setup, file on disk.

MySQL / Postgres if deploying to Render/Railway managed DB.

SQLAlchemy ORM + Alembic migrations (M3).

VECTOR STORE

ChromaDB — easiest, embedded, no server (M5 RAG master class).

FAISS — local, fast, file-based (M4).

Supabase pgvector — cloud, queryable with SQL (M4).

LLM PROVIDER

Groq (free tier, fastest) — `llama-3.3-70b-versatile` , `llama-3.1-8b-instant` .

OpenAI — `gpt-4o-mini` for chat, `gpt-4o` for vision, `text-embedding-3-small` .

Ollama — local model fallback (M4).

EMBEDDINGS

OpenAI `text-embedding-3-small` (1536-d, paid but cheap).

sentence-transformers `all-MiniLM-L6-v2` (384-d, free, local).

AGENTIC ORCHESTRATION

LangGraph — `StateGraph` , conditional edges, `InMemorySaver` , `interrupt` (M6).

Hand-rolled multi-agent — coordinator + specialists pattern (M5).

Need a real LLM inside a LangGraph node? See [ref-langgraph-llm-wiring.pdf](#).

VISION / MULTIMODAL

OpenAI `gpt-4o` with base64-encoded image input (M6).

Image generation: `gpt-image-1` (M6).

DEPLOYMENT

Streamlit Community Cloud (free) for Streamlit apps.

Render / Railway free tier for FastAPI + Postgres.

Step-by-step: [ref-deploy-render-railway.pdf](#).

PROJECT 01 · KNOWLEDGE WORK

Domain Knowledge Co-Pilot

A chat app that answers questions over your own document corpus — with citations linking back to the exact source passage.

PROBLEM

Researchers, consultants, analysts, students, and domain experts spend hours digging through their own PDFs, notes, and reports to answer the same kinds of questions. A RAG co-pilot replaces "search → open PDF → ctrl-F → read" with "ask, get answer, click citation, verify." The catch is doing retrieval well: naive top-k similarity often returns near-duplicates or misses the relevant chunk.

CORE OUTCOMES

- Users sign up / log in (JWT) and own their corpora.
- Upload PDFs / DOCX / TXT / Markdown — the app chunks, embeds, indexes.
- Chat interface — natural-language questions over the active corpus.
- Answers come with **citations**: source filename + chunk text + page (where parseable).
- Multiple corpora per user (e.g., "client A", "thesis-lit-review", "personal-knowledge").
- Per-corpus chat history saved and resumable.

SUGGESTED LAYERS

FRONTEND

Streamlit for the chat (built-in `st.chat_input`, `st.chat_message`), corpus picker in sidebar, upload widget.

Alt: vanilla HTML+JS — needs `fetch()` + CORS (see ref doc).

BACKEND / AI CORE

FastAPI endpoints: `POST /corpora`, `POST /corpora/{id}/upload`, `POST /corpora/{id}/query`. Auth via JWT.

RAG: chunk with overlap → embed → store → retrieve top-k → answer with **Groq llama-3.3-70b** or **OpenAI gpt-4o-mini**.

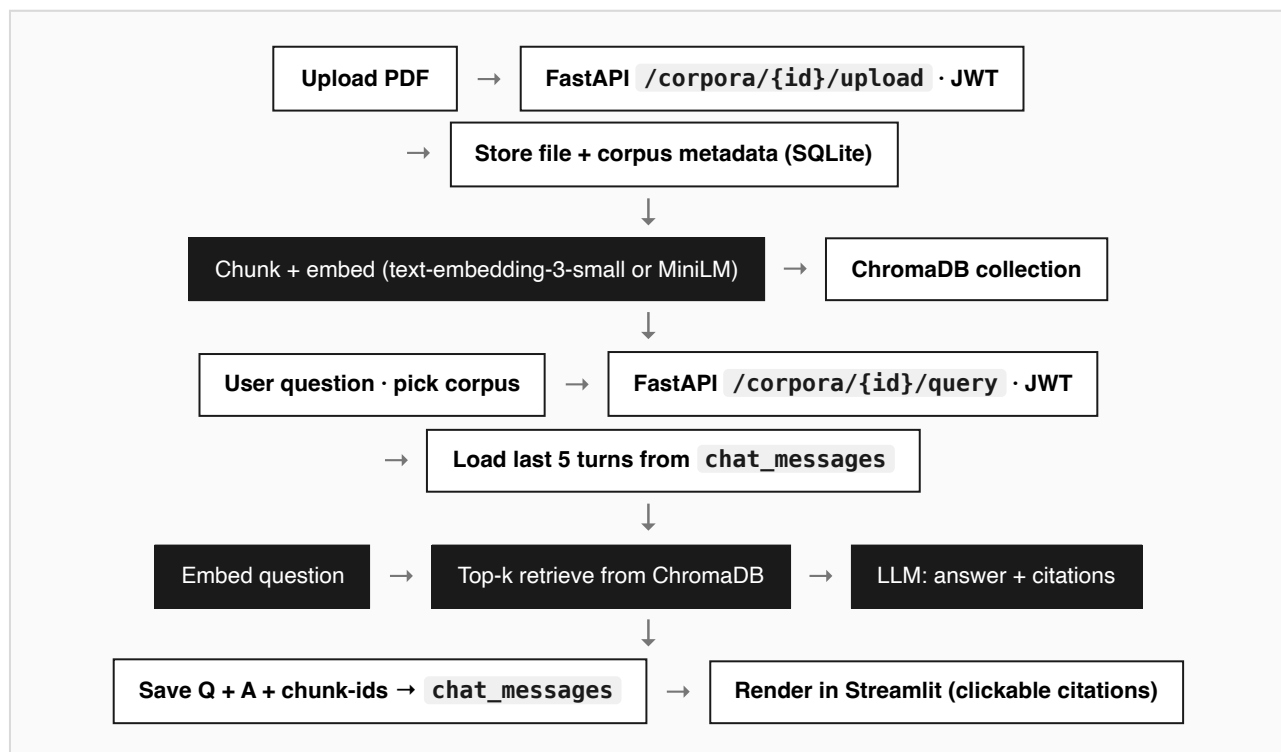
PDF parse: `pypdf` or `pdfplumber`.

STORAGE

ChromaDB for vectors (one collection per corpus). **SQLite + SQLAlchemy** for users, corpora metadata, chat history.

Alt vector: FAISS (in-process) or Supabase pgvector (cloud).

PSEUDO-WORKFLOW



SAMPLE FEATURES TO BUILD

- JWT signup / login flow (reuse your M3 student-mgmt-API code).
- Upload page: drag-drop, progress bar, parsed-chunk preview.
- Corpus picker — switch between corpora; each has its own chat history.
- Chat with conversation memory (include last 5 turns in the prompt).
- Citations rendered as numbered footnotes; clicking expands the source chunk.
- "Show retrieved chunks" debug toggle — see what the retriever found before the LLM answered.

OPEN-ENDED STRETCH

- **Hybrid retrieval** — combine BM25 keyword search (using `rank_bm25`) with semantic similarity; weighted reciprocal rank fusion.
- **Reranking** — second pass with a cross-encoder (`sentence-transformers cross-encoder/ms-marco-MiniLM-L-6-v2`) over top-20 to pick top-5.
- **Query reformulation** — first LLM call rewrites ambiguous user queries; second call answers.
- **Cross-corpus comparison** — "what does Corpus A say about X vs Corpus B?"
- Deploy with [ref-deploy-render-railway.pdf](#).

Reference material:

- [ref-fastapi-cors-fetch.pdf](#) — only if you go vanilla-JS frontend.
- [ref-deploy-render-railway.pdf](#) — for the deploy stretch.

Everything else is in your M3 + M4 + M5 sessions.

PROJECT 02 · CAREER & PERSONAL BRANDING

Job Application Co-Pilot

A multi-agent system that turns your resume + a job description into a tailored application kit: fit analysis, rewritten resume, cover letter, and a mock interview.

PROBLEM

Applying for jobs is a grind: read the JD, re-read your resume, tweak bullet points for relevance, write a fresh cover letter, prepare for likely interview questions. Every role gets ~20 minutes of effort because that's all anyone has. AI can take the grind out — turn "20 minutes of tired tweaking" into "10 minutes of reviewing four high-quality drafts."

CORE OUTCOMES

- User uploads their resume (PDF) and pastes a JD (text or URL).
- The app runs a multi-agent pipeline producing:
 - Fit analysis (which JD requirements you meet, which you don't, what to emphasise)
 - Tailored resume rewrite (same role list, sharper bullets, JD keywords woven in)
 - Cover letter draft (1 page, tone-matched to the company)
 - Mock interview question pack (10 likely questions + sample answers grounded in your resume)
- All artifacts saved per "role application" entry — user can manage a pipeline of roles applied for.
- Diff view — see resume bullet "before" and "after" side by side.

SUGGESTED LAYERS

FRONTEND

Vanilla HTML + CSS + JS — this is a CRUD app (multiple roles, drafts, diffs) and you have practice from the Task Flow Kanban. Use `fetch()` to FastAPI.

Alt: Streamlit — simpler, but multi-page navigation is clunky.

BACKEND / AI CORE

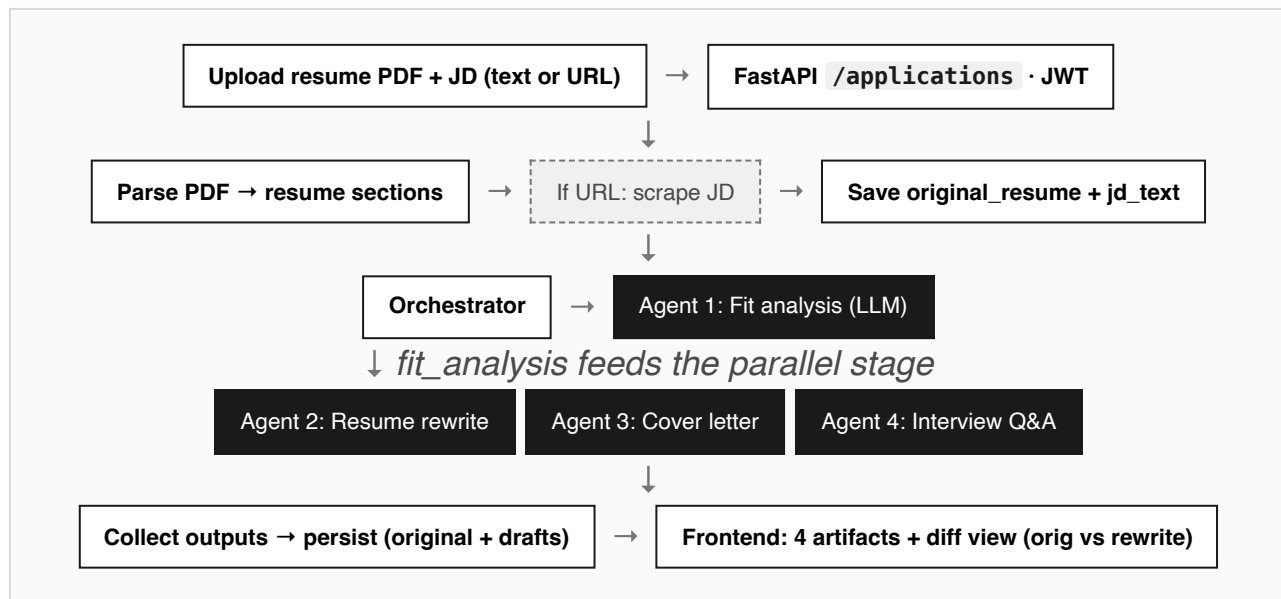
FastAPI with JWT auth.

Multi-agent pipeline: orchestrator → fit-analyst → resume-writer → cover-letter-writer → interviewer. Implement with **LangGraph supervisor pattern** (M6) or hand-rolled coordinator (M5). LLM: **Groq llama-3.3-70b** (fast, free) or **OpenAI gpt-4o-mini**.

STORAGE

SQLite + SQLAlchemy: users, roles (job_title, company, jd_text), drafts (resume_rewrite, cover_letter, interview_qa), revisions.
Resume PDF parse: `pypdf`.

PSEUDO-WORKFLOW



SAMPLE FEATURES TO BUILD

- Resume upload + PDF parsing into structured sections (header, experience, education, skills).
- JD paste box; bonus: paste a LinkedIn URL and scrape with `requests` + `beautifulsoup4`.
- "Roles" list view — every application you've prepped, with status (applied / not yet / rejected / interviewed).
- Diff view for resume bullets (old text struck out, new text in green).
- "Regenerate this section" buttons — re-run a single agent without rerunning the whole pipeline.
- Download buttons: cover letter as `.docx` (use `python-docx`), resume as PDF.

OPEN-ENDED STRETCH

- **ATS scorer** — second LLM pass scoring the rewritten resume against the JD on keyword density.
- **Voice mock interview** — use browser speech-to-text for your answer, LLM grades it, gives feedback.
- **Salary negotiation coach** — give the role + your offer, get scripts for negotiation.
- **Calendar integration** — schedule follow-up reminders 1 week after applying.
- Deploy with [ref-deploy-render-railway.pdf](#).

Reference material:

- [ref-fastapi-cors-fetch.pdf](#) — required: your vanilla JS frontend needs to call FastAPI.
- [ref-langgraph-llm-wiring.pdf](#) — only if you use LangGraph for the multi-agent pipeline.
- [ref-openai-native-tools.pdf](#) — for native function-calling if any agent needs tools (e.g., LinkedIn scrape).
- [ref-deploy-render-railway.pdf](#) — for the deploy stretch.

PROJECT 03 · PERSONAL PRODUCTIVITY

Personal Productivity Agent

A daily-log app with an LLM agent that remembers your history, surfaces what's overdue, drafts your EOD summary, and tells you what to do tomorrow.

PROBLEM

Most people don't have a productivity problem — they have a memory and prioritisation problem. They forget what was due, they forget how long things actually take, and they end every day unsure if they got the important things done. A stateful agent that you check in with twice a day (morning / evening) can be the missing layer: it reads your history, surfaces patterns, and writes the boring summary for you.

CORE OUTCOMES

- Daily check-in: user logs tasks (one-liners), notes, and what they completed today.
- Agent classifies tasks (work / personal / health / learning), tags urgency.
- Agent surfaces overdue items — anything tagged for "today" that didn't get marked complete.
- EOD summary auto-drafted — 1 paragraph "here's what you got done, here's what slipped."
- Tomorrow's plan suggested based on what slipped + priorities + your patterns.
- Weekly review (every Sunday) — patterns surfaced ("you push 'gym' to next week 4 weeks running").

SUGGESTED LAYERS

FRONTEND

Streamlit — daily check-in form, task list, "Run my EOD" button, "Show this week" view. Best fit because UI is data-light and chat-like.

Alt: vanilla HTML+JS for a more app-like feel.

BACKEND / AI CORE

FastAPI + LangGraph stateful agent (M6).

Nodes: classifier → overdue-surfacers → EOD-summarizer → next-day-planner.

Checkpointers: **SQLite**-backed (durable across runs) — see ref doc.

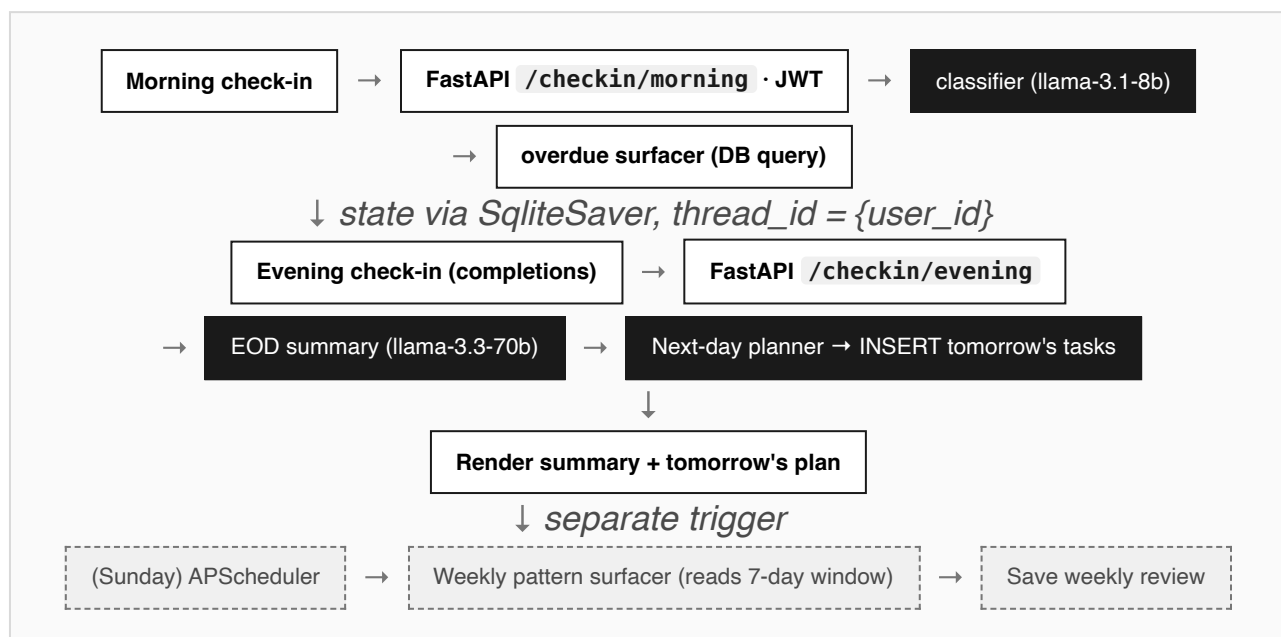
LLM in nodes: **Groq llama-3.1-8b** for cheap classification, **llama-3.3-70b** for summaries.

STORAGE

SQLite + SQLAlchemy: `users`, `tasks` (`due_date`, `priority`, `category`, `completed_at`), `daily_logs`, `eod_summaries`.

LangGraph `thread_id` = `{user_id}` (one rolling thread per user; `SqliteSaver` carries state forward).

PSEUDO-WORKFLOW



SAMPLE FEATURES TO BUILD

- Auth — JWT signup/login.
- Morning check-in form: "what are you planning today?" → free text + structured task entry.
- Evening check-in: "what did you actually do?" → mark tasks complete, add anything that emerged.
- Task list view — colour by category, group by urgency.
- "Run my EOD" — triggers the LangGraph pipeline; saves the generated summary.
- "This week" view — shows daily summaries side by side.
- Pattern surfacer: scheduled every Sunday — what tasks did you repeatedly push? What categories dominated?

OPEN-ENDED STRETCH

- **Voice morning check-in** — browser speech-to-text; LLM extracts structured tasks from your rambling.
- **Calendar import** — read Google Calendar events as additional context (mock JSON file if real auth is too much).
- **Streak gamification** — track consecutive days of check-ins; LLM congratulates.
- **Smart reminders** — daily scheduled job (APScheduler) pings you with overdue items via email.
- Deploy with [ref-deploy-render-railway.pdf](#).

Reference material:

- [ref-langgraph-llm-wiring.pdf](#) — **required**. Your M6 LangGraph demos used hardcoded responses; this shows the SQLite checkpointer + wiring Groq/OpenAI into nodes.
- [ref-cron-scheduling.pdf](#) — for the weekly pattern-surfacers and daily reminder stretch.
- [ref-fastapi-cors-fetch.pdf](#) — only if you go vanilla-JS frontend.
- [ref-deploy-render-railway.pdf](#) — for the deploy stretch.

PROJECT 04 · MULTIMODAL / HEALTH-OF-THINGS

Plant Doctor

Upload a photo of a sick houseplant; the app identifies the species, diagnoses the issue, prescribes a recovery plan, and tracks how the plant is doing week-over-week.

PROBLEM

People love their houseplants and feel terrible when they're dying. Diagnosing what's wrong — overwatering vs underwatering vs sunlight vs pests vs nutrient deficiency — usually means a 45-minute Reddit deep-dive. A vision model can look at the photo, ask a couple of targeted questions, and prescribe a fix. A stateful app can then check in weekly: "show me a new photo, here's what changed, adjust the plan."

CORE OUTCOMES

- User signs up and registers their plants ("Pothos in living room, north-facing window").
- Per plant: upload a photo + describe what's wrong.
- Vision model identifies species (if missing), diagnoses issue category (water / light / pest / nutrient / disease).
- Conversational follow-up — agent asks 2-3 clarifying questions ("how often do you water?", "any new leaves in the last week?") before prescribing.
- Care plan generated: step-by-step actions, expected recovery time, what to look for in 1 week.
- Weekly check-in: upload a new photo; agent compares to last photo, updates the plan.

SUGGESTED LAYERS

FRONTEND

Streamlit — file_uploader is one line, image display is one line, chat is one line. Heavy image UI in vanilla JS is a lot of plumbing.

Alt: vanilla — only if you've done image upload before.

BACKEND / AI CORE

FastAPI + OpenAI gpt-4o (vision) for image-in.

LangGraph stateful agent
(thread_id = plant_id): identify → diagnose → ask-clarifying-Qs → prescribe → followup.

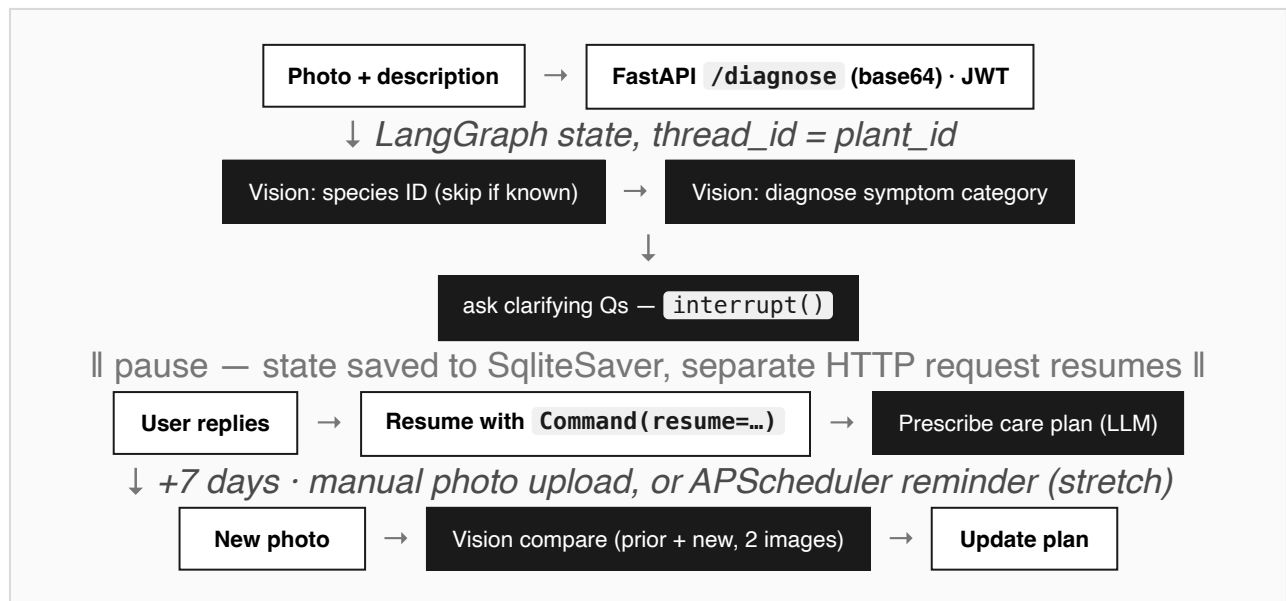
Free vision alt: Groq's meta-llama/llama-4-scout-17b-16e-instruct (preview, free tier).

STORAGE

SQLite + SQLAlchemy: users, plants (species, location, planted_at), photos (filepath, taken_at, plant_id), diagnoses, care_plans.

Image files on local disk under uploads/{user_id}/{plant_id}/.

PSEUDO-WORKFLOW



SAMPLE FEATURES TO BUILD

- JWT auth.
- "My plants" dashboard — cards with last photo + status (healthy / recovering / critical).
- Upload + diagnose flow with image preview and progress.
- Conversational follow-up Qs (use LangGraph `interrupt` — M6) before prescribing.
- Care plan rendered as a checklist; user marks items done.
- "Weekly check-in" — guided photo upload + 1-line update; agent surfaces "is the leaf yellowing reducing?" comparisons.
- Photo gallery per plant — visual recovery timeline.

OPEN-ENDED STRETCH

- **RAG over plant-care knowledge base** — ingest care guides (Reddit r/houseplants top posts, UC IPM pest guides) into ChromaDB; ground answers with citations.
- **Side-by-side photo diff** — render last vs current photo with overlaid notes from the agent.
- **Reminder scheduler** — APScheduler nudges user when a watering / fertilising step is due.
- **Plant collection — community variant** — public anonymized gallery of "successfully recovered" cases by symptom.
- Deploy with [ref-deploy-render-railway.pdf](#).

Reference material:

- [ref-langgraph-llm-wiring.pdf](#) — **required**. Wiring real OpenAI vision into LangGraph nodes + SQLite checkpoint for "1 week later" continuation.
- [ref-cron-scheduling.pdf](#) — for the reminder stretch.
- [ref-deploy-render-railway.pdf](#) — for the deploy stretch.

Vision API usage is in your M6 multimodal session.

PROJECT 05 · CONTENT / PERSONALISATION

Personal Newsletter Curator

An agent that fetches stories from RSS feeds / HN / Reddit on the topics you care about, summarises them, and explains "why this matters to you" — emailed daily or weekly.

PROBLEM

Information overload is real. Most people give up on RSS, leave 1,247 unread emails in their newsletter folder, and end up doomscrolling X for "what's important today." A curator agent that knows your interests, ranks new stories by relevance, summarises in your voice, and tells you specifically why each story matters to *you* — that's the unlock. Bonus: as you click some stories and ignore others, it learns.

CORE OUTCOMES

- User signs up and tells the app their interests (topics, keywords, people / companies / projects to track).
- Daily (or user-chosen cadence) — agent runs: pulls feeds → relevance-ranks against the user's interest model → summarises top stories.
- Personalisation layer — past clicks influence future ranking ("you clicked everything about local-first software; I'll surface more").
- Each story shows: 2-sentence summary + 1-sentence "why this matters to you" + source link.
- In-app digest viewer + optional email delivery.

SUGGESTED LAYERS

FRONTEND

Streamlit — digest viewer, interest editor, click tracker. Easy and fast.

Alt: vanilla HTML+JS, especially if you want a slick reading view.

BACKEND / AI CORE

FastAPI with **APScheduler** for daily/weekly fetch jobs.

Pipeline: **fetcher** (RSS via `feedparser` , HN via Algolia API, Reddit via `praw`) → **relevance ranker** (embedding similarity vs user interest text) → **summariser** (Groq) → **personaliser** (LLM + click history context).

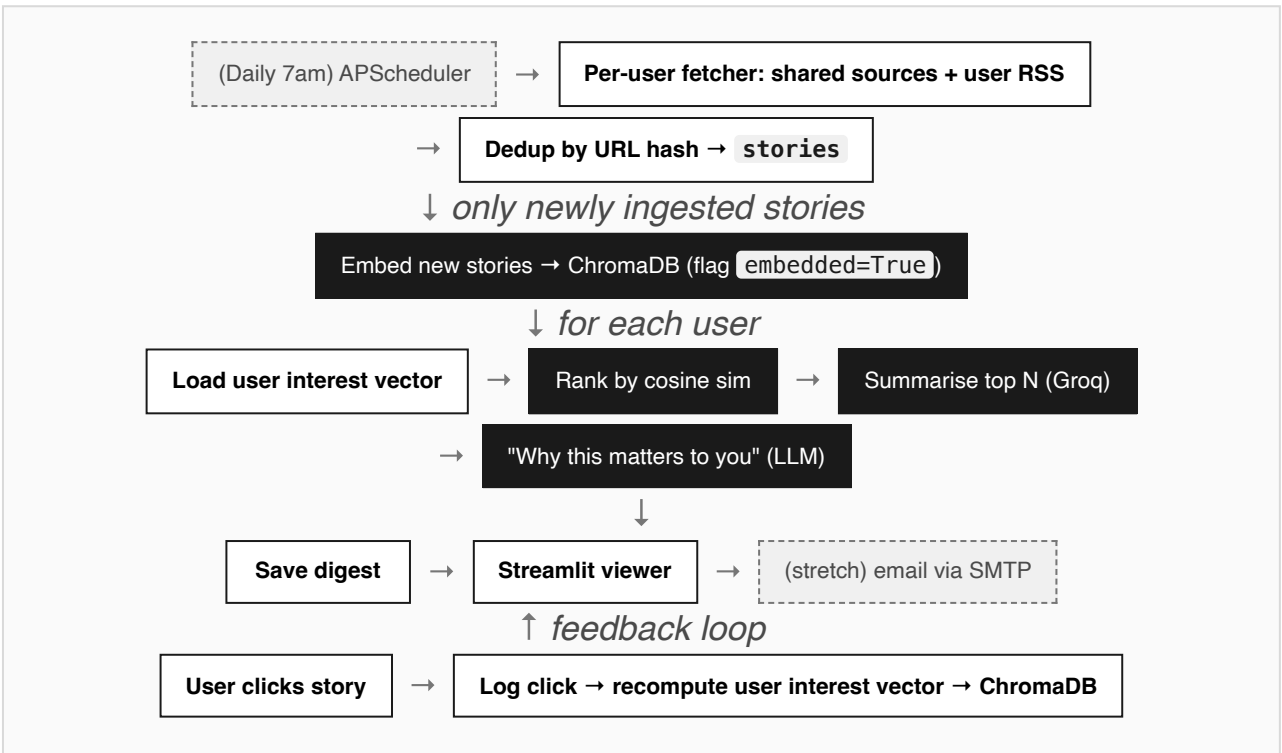
Hand-rolled multi-agent (M5 pattern) is fine; LangGraph also works.

STORAGE

SQLite + SQLAlchemy: `users` , `interests` , `sources` , `stories` , `clicks` , `digests` .

ChromaDB for interest embeddings + story embeddings (cosine similarity for relevance ranking).

PSEUDO-WORKFLOW



SAMPLE FEATURES TO BUILD

- Interest editor — free-text "what do I care about?" + structured topic tags.
- Source manager — add/remove RSS feeds, subreddits, HN tags.
- Daily digest view — card per story (summary + "why" + click-through).
- Click tracker — record clicks; update interest embedding.
- "Trending in your topics" panel — stories with high engagement across feeds.
- Manual run button — fetch + curate now (don't wait for the scheduler).

OPEN-ENDED STRETCH

- **Email delivery** — SMTP via Gmail app password; HTML email template with the daily digest.
- **Audio version** — TTS (OpenAI `tts-1`) reads the digest; downloadable MP3 — your personal podcast.
- **Cross-source dedup** — story embeddings; merge near-duplicates from different sources.
- **Discovery mode** — once a week, surface 1-2 stories *outside* your usual topics that share patterns with what you've clicked. Anti-filter-bubble.
- Deploy with [ref-deploy-render-railway.pdf](#).

Reference material:

- [ref-cron-scheduling.pdf](#) — **required**. APScheduler integration with FastAPI for daily fetch jobs.
- [ref-openai-native-tools.pdf](#) — only if you build a "ask the curator a question" agent with tool access.
- [ref-deploy-render-railway.pdf](#) — for the deploy stretch.